Real-Time Sui

GraphQL Subscriptions and the Data Stack After JSON-RPC

01 / CHOOSE

Map the data stack

02 / BUILD

A feed that survives a drop

03 / MIGRATE

One call in your own app

A payment screen is the worked example for the whole lesson.

Replace the data path your app depends on.

Start with what the screen or worker needs to do.

APP NEED	REPLACEMENT PATH	TILL EXAMPLE
State + execution	gRPC	Read balances; submit a payment
Filtered activity	GraphQL query + subscription	Load receipts; receive new ones
Pruned history	Archive / backed GraphQL	Find an old transaction by digest

July: Foundation mainnet shutoff → September: GraphQL subscriptions → Migrate each call

Where each piece sits in the data stack.

SOURCE	DATA SERVICES	APP DATA VIEW	CONSUMER
Sui	gRPC + archive	GraphQL	Your app
Finalized checkpoints	Live data + retained history	Filter · select · stream	Screen or small DB writer

OWNED PIPELINE

Checkpoint data → Rust indexing framework → Your schema + database

Wallet: state + activity Shop: filtered receipts
Game: actions + domain events Analytics: custom checkpoint processing

Till: follow one payment all the way through.

CUSTOMER

Send 1 USDC

Pays SUI gas

→

CHAIN

Payment finalizes

Shop gains 1 USDC

→

LIVE SCREEN

Receipt + chime

Receives the delivery

→

DURABLE LEDGER

Save + total

Keep the receipt

Then disconnect, recover, total the day and look up an old receipt.

Target: USDC. Current live app: SUI verified; a funded USDC walkthrough is still pending.

Filter for the activity your app needs.

STREAM	FILTER	EXAMPLE USE
transactions	affectedAddress	Any transaction touching this shop
transactions	affectedObject	Activity involving a game object
transactions	function	Calls to a Move package / module / function
events	sender	Events from transactions sent by an address
events	module	Events emitted by a particular module
events	type	One domain event, such as a sale event

Till uses transactions + affectedAddress. A match still needs payment validation.

Current schema: choose module or type for an event filter; combining them returns an error.

A matching transaction becomes a receipt.

```
subscription Till($shop: SuiAddress!,
 $after: String, $afterCheckpoint: UInt53) {
 transactions(filter: { affectedAddress: $shop,
   afterCheckpoint: $afterCheckpoint }, after: $after) {
   cursor
   node { digest sender { address }
     effects { status timestamp
       checkpoint { sequenceNumber }
       balanceChanges(first: 50) { nodes {
         owner { address } coinType { repr } amount
       }}
     }
   }
 }
}
```

CURSOR

Where to resume delivery

DIGEST + STATUS

Transaction + outcome

OWNER + COIN + AMOUNT

Net credit to this shop

TIMESTAMP + CHECKPOINT

When the receipt belongs

SUCCESS + positive net credit
→ receipt → render + chime

One POST request. Many SSE deliveries.

```
const res = await fetch(url, {
 method: 'POST',
 headers: {
   'content-type': 'application/json',
   accept: 'text/event-stream',
 },
 body: JSON.stringify({ query, variables }),
 signal,
});
```

REQUEST

POST /graphql/subscriptions
query + variables

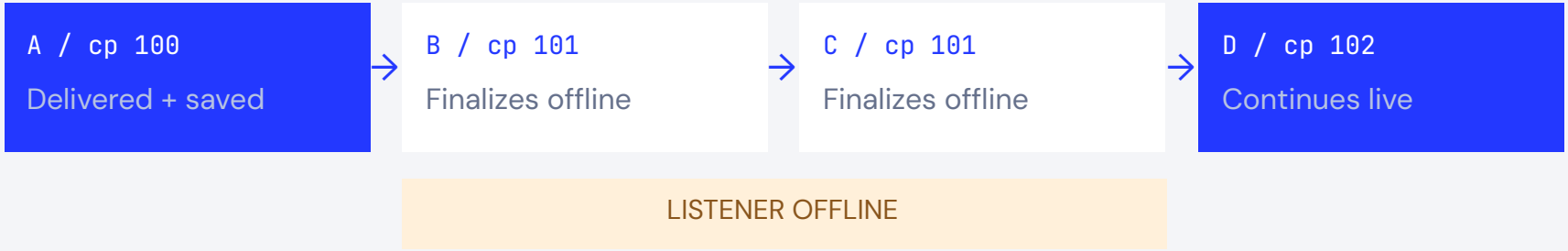
RESPONSE STAYS OPEN

text/event-stream
receipt A → B → C → …

fetch → decode text → buffer complete SSE frames → inspect errors → await the handler

EventSource cannot send this POST body. A network chunk is not a complete SSE message.

The connection drops. The payments do not.



Saved cA

Reconnect: after = cA

B → C → D

No cursor yet? Save the opening checkpoint, then use filter.afterCheckpoint.

Order: checkpoint → transaction position → event position. Persist before advancing.

Replay safely. Count each receipt once.

```
db.exec('BEGIN');
for (const p of payments) {
 const r = insert.run(p.digest, p.coinType,
   p.amount, p.sender, p.checkpoint,
   p.timestamp, p.cursor);
}
saveCursor.run(shop, d.transactions.cursor,
 new Date().toISOString());
db.exec('COMMIT');
```

TRANSACTION BODY EXCERPT / ERROR PATH IN indexer/indexer.ts

DEDUPLICATE

Unique (digest, coin_type)

COMMIT ATOMICALLY

Receipt rows + saved cursor

ON FAILURE

Roll back both; reconnect

pnpm indexer → stop → pay → restart → pnpm totals 2026-10-16

Totals come from durable rows, grouped by coin and UTC day, using BigInt base units.

When does a small writer stop being enough?

TILL / SMALL APPLICATION FEED

GraphQL → TypeScript → DB

One shop or selected event types
A few derived tables and totals
Restart from a committed cursor

ANALYTICS / OWNED PIPELINE

Checkpoints → Rust → DB

Broad backfills and reprocessing
Custom schemas and joins
Control throughput and retention

Decide from coverage, processing cost and operational ownership; measure before scaling.

An old receipt needs retained history.

ONE KNOWN TESTNET TRANSACTION

`ArCwXsZUTywBuM4GbNX4CVMERmBBgeYY2mgmJJjdHa3a`

PUBLIC FULL NODE / VERIFIED 15 OCT

Not found

This endpoint did not return the old record.

→

GRAPHQL / VERIFIED 15 OCT

SUCCESS · 7 October 2025

Checkpoint 250,000,000
Archive-backed history path

pnpm receipt / Full node retention and archival history serve different needs.

Build for limits, reconnects and changes.

WHAT CAN FAIL	FAILURE / LIMIT	WHAT TO DO
Query limits	5,000 B / 40 s; pages 20 / 50	Select less; page nested collections
Subscriber pressure	Throttling / stream disconnects	Bound work; back off; resume cursor
Account / query	Old cursor may no longer fit	Scope state; stop stale connections
Address deposits	Credit arrived after catch-up	Verify digest; reconcile missed data
Prototype persistence	Browser has only 500 receipts	Keep a durable writer + database

Before launch: use a provider or operate the service; verify quotas, retention and stream support.

Also: HTTP 200 can contain errors; public faucets can delay; clearing site data loses demo keys.

Lab: prove each part of the payment journey.

ACTION	EXPECTED EVIDENCE	PART OF THE STACK
Open + pay	One receipt + chime; shop credit	gRPC execution + GraphQL feed
Pause → pay → resume	Missed digest appears	Cursor backfill → live
Reload / restart writer	Same receipt counted once	Deduplication + durable cursor
pnpm totals	DB total for the chosen UTC day	SQLite writer + BigInt
Find old receipt	Date + checkpoint + SUCCESS	Archive-backed lookup

TESTNET / VERIFIED PAYMENT PATH: SUI / USDC RUN STILL PENDING

day3-till.vercel.app

Repo: github.com/Nuel-osas/day3-till

Migrate the client, then check the response.

BEFORE / migration/legacy-jsonrpc.ts

```
import { SuiJsonRpcClient,
 getJsonRpcFullnodeUrl
} from '@mysten/sui/jsonRpc';

const client = new SuiJsonRpcClient({
 url: getJsonRpcFullnodeUrl('testnet'),
 network: 'testnet'
});

client.getBalance({ owner: SHOP })
```

AFTER / migration/migrated-grpc.ts

```
import { SuiGrpcClient }
 from '@mysten/sui/grpc';

const client = new SuiGrpcClient({
 network: 'testnet',
 baseUrl:
   'https://fullnode.testnet.sui.io:443'
});

const { balance } =
 await client.getBalance({ owner: SHOP });
```

Check: result shape + total balance + errors + pagination. Obtain fresh API cursors.

Your app: replace one call and prove it works.

01	Find one call	What does its screen need?
02	Choose + replace	gRPC, GraphQL or a subscription
03	Verify the behaviour	Known data + one failure case

Leave with a working diff, its evidence, and a reason for your data-stack choice.

github.com/Nuel-osas/day3-till

day3-till.vercel.app